

Leçon : structures de données

Rappels (types construits)

- **Les listes.** Exemple : notes = [10, 11, 13]

Parcours par valeur

Parcours par indice

- **Les tuples**

Les **tuples** (appelés **p-uplets**) sont une collection d'objets ordonnée mais

On dit que ce sont des objets ou immutables, on dit aussi parfois non mutables (tous ces termes sont synonymes).

→ Accès aux coordonnées (avec l'affectation multiple) : point = (1,2)

```
x, y = point  
  
print(x)    # 1  
print(y)    # 2
```

- **Les dictionnaires**

Un dictionnaire est constitué de paires non ordonnées **clé** : **valeur**. Chaque **clé** est présente de façon **unique** dans le dictionnaire. Exemple :

```
ferme_gaston = {"lapin": 5, "vache": 7, "cochon": 2}
```

- ferme_gaston.**keys()** permet d'accéder à toutes les **clés** de ferme_gaston
- ferme_gaston.**values()** permet d'accéder à toutes les **valeurs** de ferme_gaston
- ferme_gaston.**items()** permet d'accéder à tous les **couples (clé, valeur)** de ferme_gaston

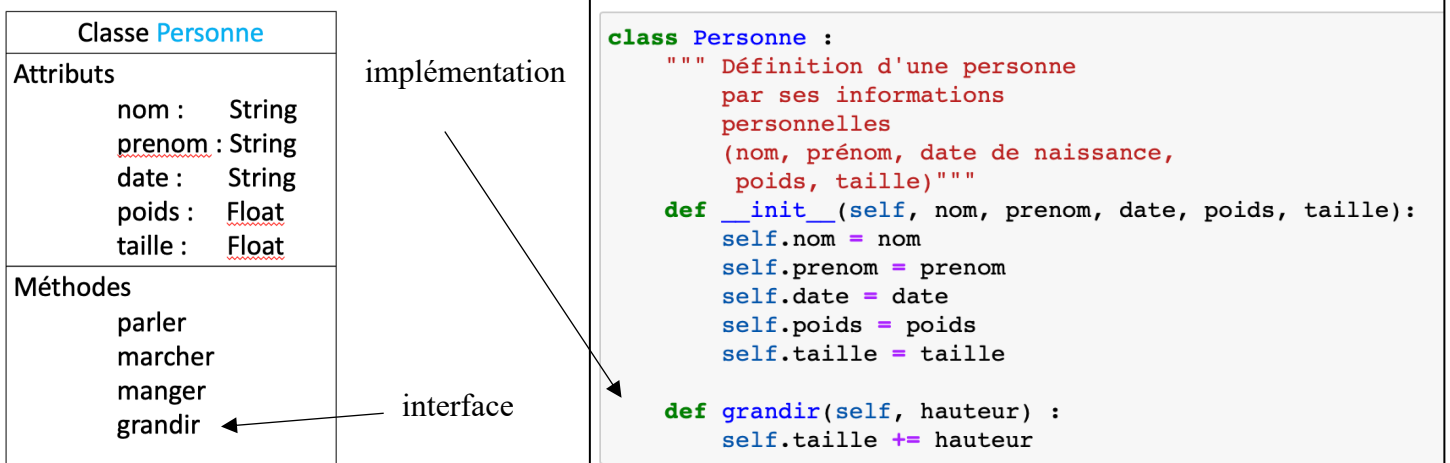
Programmation Orientée Objet (POO)

Les modules indépendants du programme sont des classes. Une **classe** regroupe les caractéristiques et les comportements communs des objets. Un **objet** est un élément de la classe ou une **instance** de la classe.

Exemple : la classe Personne et les instances (ou objets) sont Alice, Pierre...

Un objet est défini par :

- Ses **attributs** ou champs (variables qui caractérisent l'état qui varie au cours du temps pendant l'exécution du programme).
- Ses **méthodes** (opérations ou fonctions que l'on peut appliquer sur l'objet qui déterminent le comportement des objets et qui peuvent modifier les attributs d'un objet).



La méthode **__init__** est le **constructeur** de la classe. Elle permet de créer les objets (ou les **instances**). Le 1^{er} paramètre est toujours self, l'objet lui-même sur lequel la méthode s'applique.

Interface / implémentation

- **L'interface** indique ce que l'on peut faire avec un objet
- **L'implémentation** décrit comment l'objet réalise ces actions.

Accès aux attributs et aux méthodes

Les méthodes prennent `self` en 1^{er} paramètre. La notation **point .** permet d'accéder aux attributs et aux méthodes d'un objet.

```
help(Personne)
```

```
elio = Personne("Nicosia", "Elio", "11122010", 40, 1.45)
taille_dateT1 = elio.taille
elio.grandir(0.05)
taille_dateT2 = elio.taille

print("Elio mesure ", taille_dateT2, "m après avoir grandi de 5 cm")
print("Avant, il meserait : ", taille_dateT1, "m")
```

```
Elio mesure 1.5 m après avoir grandi de 5 cm
Avant, il meserait : 1.45 m
```

La méthode `str` (fait partie des méthodes particulières en Python encadrées par `__`). Elle renvoie une chaîne de caractères décrivant l'objet. L'appel avec `print` permet d'afficher l'objet.

```
def __str__(self):
    return f"{self.prenom} {self.nom} pèse {self.poids} kg et mesure {self.taille} m."
```

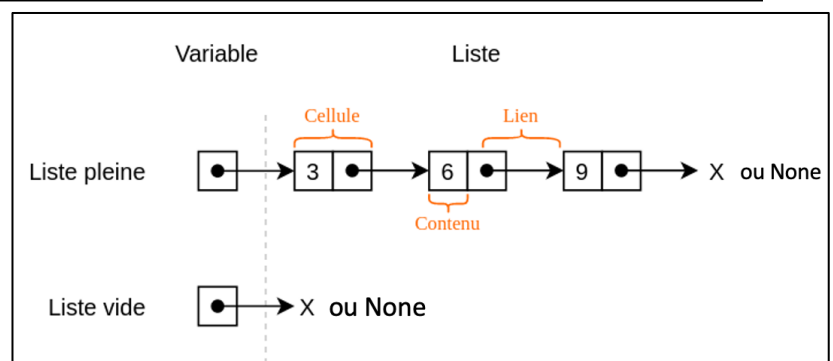
```
print(elio)
```

```
Elio Nicosia pèse 40 kg et mesure 1.5 m.
```

Les listes chaînées

La liste est chaînée lorsque la liste fait apparaître une chaîne de valeurs : chacune pointant vers la suivante.

Chaque **cellule** contient une **valeur** et un lien (appelé **pointeur**) vers la suivante (adresse mémoire où se trouve la cellule contenant l'élément suivant de la liste).



La classe `Liste` possède un unique attribut `tete` qui désigne la tête de la liste lorsque celle-ci n'est pas vide (et `None` sinon). Le constructeur initialise l'attribut `tete` avec la valeur `None`.

Cellule	Liste
Valeur	tete = None
suivante	est_vide() ajoute(x) __len__() __str__()

```
class Cellule:
    def __init__(self, valeur, suivante=None):
        self.valeur = valeur
        self.suivante = suivante
```

```
class Liste:
    """une liste chaînée"""
    def __init__(self):
        self.tete = None
```

Quelques méthodes au sein de la classe Liste :

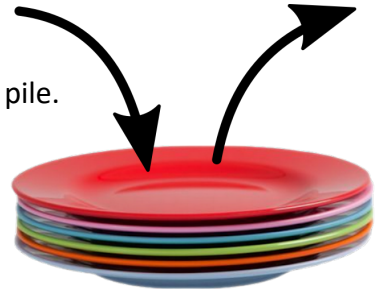
- `est_vide()` : renvoie True si la liste est vide et False sinon.
- `ajoute(x)` : modifie l'attribut `tete` en ajoutant la valeur `x` en tête.
- `queue()` : renvoie la liste en enlevant la tête.
- `__len__()` : renvoie le nombre d'éléments de la liste.
- `__str__()` : affiche le contenu de la liste.

Les piles

Une pile (stack en anglais) est un ensemble ordonné d'éléments qui se comporte comme une pile d'assiettes.

Les éléments sont empilés les uns au-dessus des autres

- On peut ajouter une assiette sur la pile ou prendre l'assiette sur le dessus de la pile.
- C'est la méthode LIFO (Last In, First Out).
- Le dernier élément à être arrivé est donc le premier à être sorti.



Les files

- Une file (ou queue en anglais) est un ensemble ordonné d'éléments qui se comporte comme une file d'attente.
 - Un nouvel arrivant se met à la fin de la file tandis que la prochaine personne servie est celle en tête de file.
 - Le premier élément à être arrivé est donc le premier à en sortir.
 - C'est la méthode FIFO (First in, First Out).
 - Exemple :
 - Les documents envoyés à l'imprimante sont traités dans une file d'impression.
- C'est le premier document envoyé à l'imprimante qui va être imprimé en premier.

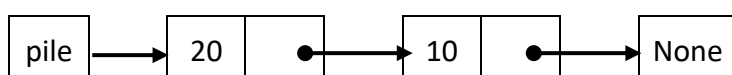
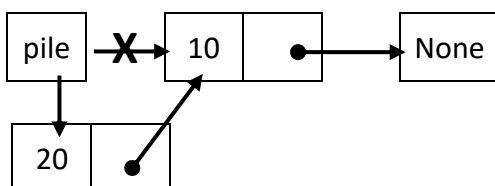


Chacune des 2 structures (**Pile** ou **File**) a une **interface** proposant au minimum les 4 opérations suivantes :

- **Créer** une structure initialement vide
- Tester si une structure **est vide**
- **Ajouter** un élément à une structure (**empiler** au sommet de la pile ou **enfiler** à la fin de la file)
- **Retirer** le sommet de la pile « **dépiler** » (si elle n'est pas vide) ou l'élément en début de file « **défiler** ».
- Obtenir un élément d'une structure (le **sommet** de la pile ou le **premier élément** de la file si elle n'est pas vide).

Définir la classe Pile en réutilisant la classe Cellule

Exemple d'utilisation :



```
class Pile :  
    """La classe Pile en réutilisant la classe Cellule"""  
    def __init__(self):  
        self.sommet = None  
  
    def est_vide(self):  
        """Renvoie True si la pile est vide."""  
        return self.sommet is None  
  
    def empiler(self, valeur):  
        """Ajoute une valeur au sommet de la pile."""  
        self.sommet = Cellule(valeur, self.sommet)
```

```
pile = Pile()  
  
pile.empiler(10)  
pile.empiler(20)
```

Définition d'une File par une classe :

```
class File:
    """Une file utilisant une classe et insérant les éléments de la file à la fin de la liste.
    La file 4 / 1 / 5 / 3 correspond à data = [3,5,1,4]"""

    def __init__(self):
        self.data = []

    def est_vide(self):
        """Renvoie True si la file est vide."""
        return len(self.data) == 0

    def enfiler(self, x):
        """Ajoute une valeur en fin de file."""
        self.data.append(x)

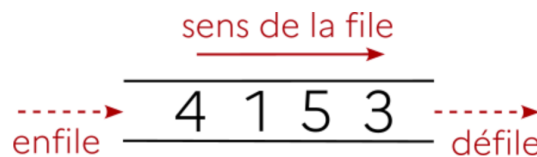
    def defiler(self):
        """Retire et renvoie la valeur en tête de file (ie. le premier élément de la liste).
        pop(0) supprime le premier élément de la liste et entraîne un décalage
        de éléments vers la gauche"""
        if self.est_vide():
            print("Impossible de défiler une file vide")
            return None
        return self.data.pop(0)

    def __str__(self):
        # pour afficher
        texte = ""
        # convenablement la file avec print
        for i in self.data :
            texte = str(i) + '|' + texte
        return texte
```

La représentation **la plus courante** d'une file se fait horizontalement, en enfilant par la gauche et en défilant par la droite :

```
f = File()
f.enfiler(3)
f.enfiler(5)
f.enfiler(1)
f.enfiler(4)
print(f)
print("data[0] = ", f.data[0])
f.defiler()
print(f)

4|1|5|3|
data[0] = 3
4|1|5|
```

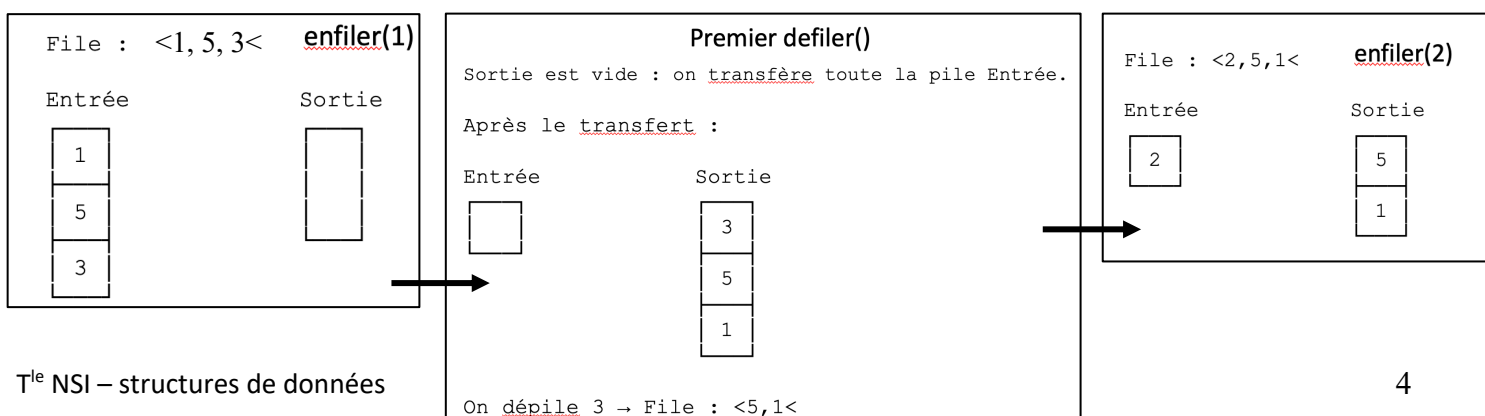


Remarque : La méthode `defiler()` agit en temps linéaire $O(n)$ et non pas en temps constant $O(1)$.

L'instruction `pop(0)` provoque un redimensionnement de la liste. Le tableau doit être réduit et chaque élément doit être recopié dans la case précédente. Ce décalage coûte un temps linéaire en fonction du nombre d'éléments de la liste.

Définition d'une File par deux piles (cf. TP) La solution pour obtenir un temps constant $O(1)$ est de créer une file avec deux piles : une pile d'entrée et une pile de sortie :

- Quand on veut enfiler, on empile sur la pile d'entrée en utilisant `append()` qui ajoute en fin donc $O(1)$
 - Quand on veut défiler, on dépile sur la pile de sortie en utilisant `pop()` qui supprime le dernier élément de la liste sans décalage avec un coût constant $O(1)$
 - Si la pile de sortie est vide, on dépile la pile d'entrée dans la pile de sortie.
- Cela inverse leur ordre et permet de récupérer le premier élément de la file.



Autre implémentation de la classe File avec un tableau où l'ordre des éléments est inversé. Ici, on ajoute au début de la liste `insert(0, x)` et on supprime le dernier élément `pop()`. C'est `insert(0, x)` qui provoque le décalage des éléments et le temps linéaire $O(n)$

Remarque : La méthode `enfiler()` agit en temps linéaire $O(n)$ et non pas en temps constant $O(1)$. L'instruction `self.data.insert(0,x)` provoque un redimensionnement de la liste. Le tableau doit être agrandi et chaque élément doit être recopié dans la case suivante. Ce décalage coûte un temps linéaire en fonction du nombre d'éléments de la liste.

Définition d'une File par une classe :

```
class File:
    """Une file utilisant une classe et insérant la suite des éléments de la file au début
    de la liste donc la file 4 / 1 / 5 / 3 correspond à data = [4,1,5,3]."""

    def __init__(self):
        self.data = []

    def est_vide(self):
        """Renvoie True si la file est vide."""
        return len(self.data) == 0

    def enfiler(self, x):
        """Ajoute la valeur x en fin de file (indice 0).
        La fin de file étant le début de liste data """
        self.data.insert(0,x)
        # self.data.append(x) ne peut être utilisé, il ajoute x à la fin de liste

    def defiler(self):
        """Retire et renvoie la valeur en tête de file (ie. le dernier élément de la liste)
        pop() sans paramètre supprime le dernier élément de la liste"""
        if self.est_vide():
            print("Impossible de défiler une file vide")
            return None
        return self.data.pop()

    def __str__(self):
        # pour afficher
        # convenablement la file avec print
        texte = '|'
        for i in self.data :
            texte = texte + str(i) + '|'
        return texte
```

```
f = File()
f.enfiler(3)
f.enfiler(5)
f.enfiler(1)
f.enfiler(4)
print(f)
print("data[0] = ", f.data[0])
f.defiler()
print(f)

|4|1|5|3|
data[0] = 4
|4|1|5|
```

Remarque : La méthode `enfiler()` agit en temps linéaire $O(n)$ et non pas en temps constant $O(1)$.

L'instruction `self.data.insert(0,x)` provoque un redimensionnement de la liste. Le tableau doit être agrandi et chaque élément doit être recopié dans la case suivante. Ce décalage coûte un temps linéaire en fonction du nombre d'éléments de la liste.